

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería · Carrera de Ingeniería de Sistemas e Informática

TALLER DE PROGRAMACIÓN WEB *Trabajo Final*

ClinicaWeb

Sitio web responsivo e interactivo para una clínica de servicios médicos

INTEGRANTES

Nombre completo	Código
Adriano Joao Souza Reyna	U24235100
Kevin Asencios Sebastian	U24263635
Jesus Miguel Canchanya Soto	U24210146
Renzo Steven Contreras Quezada	U23323758
Yosselyn Aracelly Nuñez Vasquez	U24224209
Italo Johan Barreto Villanueva	U19309695

DOCENTE

Mg. Jesús Antonio Pacahuala Arroyo

Lima – Perú
2026

Índice

1. Introducción	2
2. Planteamiento del Problema	2
3. Objetivos	2
3.1. Objetivo general	3
3.2. Objetivos específicos	3
4. Marco Teórico	3
4.1. HTML5 y semántica web	3
4.2. CSS3, Grid y Flexbox	4
4.3. Diseño responsivo y unidades relativas	4
4.4. Animaciones y transiciones CSS	4
4.5. JavaScript y manipulación del DOM	4
4.6. Arrays paralelos y funciones de arrays	4
4.7. Validación y procesamiento de formularios	4
5. Implementación de la Solución	5
5.1. Arquitectura y estructura de carpetas	5
5.2. Stack y herramientas	5
5.3. Sistema de diseño (tokens.css)	5
5.4. Inventario de páginas	6
5.5. Capa de datos: arrays paralelos (data.js)	6
5.6. Comportamiento compartido: menú hamburguesa y modales (ui.js)	6
5.7. Tablas dinámicas y filtros	6
5.8. Formularios: validación, calculadoras y selects dependientes	6
5.9. Dashboard de indicadores	7
6. Resultados y Pruebas	7
6.1. Validación de código	7
6.2. Pruebas funcionales de formularios	7
6.3. Pruebas automatizadas con jsdom	7
6.4. Pruebas de adaptación responsiva	7
6.5. Pruebas de navegación interna	7
6.6. Despliegue	8
7. Conclusiones	8
8. Recomendaciones	8
9. Bibliografía	8
10. Anexos	9

1. Introducción

El presente documento constituye el informe técnico del proyecto **ClinicaWeb**, un sitio web responsivo e interactivo desarrollado en el marco del curso Taller de Programación Web de la Universidad Tecnológica del Perú. El proyecto consiste en la implementación de un sitio web institucional para una clínica de servicios médicos ubicada en San Isidro, Lima, utilizando las tres tecnologías fundamentales del desarrollo front-end estándar: **HTML5**, **CSS3** y **JavaScript**.

En el sector salud, la presencia digital se ha convertido en un factor crítico para la accesibilidad de los pacientes a la información clínica, el agendamiento de citas y el conocimiento del cuerpo médico. De acuerdo con la Organización Panamericana de la Salud (OPS, 2022), más del 72 % de los usuarios de internet en América Latina busca información de salud en línea antes de acudir a una consulta presencial. Este comportamiento refuerza la necesidad de contar con sitios web bien estructurados, accesibles, con tiempos de carga reducidos, con información veraz y — sobre todo — con interactividad que resuelva tareas concretas del usuario.

ClinicaWeb se compone de siete páginas HTML organizadas en una arquitectura de carpetas semánticas, un sistema de diseño centralizado en variables CSS, tres formularios con validación combinada (HTML5 + JavaScript), cinco tablas de datos accesibles, un menú de navegación responsivo con botón hamburguesa, ventanas flotantes (modales), calculadoras en tiempo real y un panel de indicadores (*dashboard*) que procesa datos con JavaScript. Toda la capa dinámica se construyó en JavaScript puro (*vanilla*), sin frameworks ni dependencias externas, y el sitio se despliega de forma automática en GitHub Pages.

El documento se estructura en diez secciones: introducción, planteamiento del problema, objetivos, marco teórico, implementación de la solución, resultados y pruebas, conclusiones, recomendaciones, bibliografía y anexos. Cada sección busca evidenciar el cumplimiento de los requisitos académicos establecidos en la rúbrica del proyecto y demostrar la aplicación de buenas prácticas modernas de desarrollo web front-end.

2. Planteamiento del Problema

Las clínicas medianas y pequeñas en Lima Metropolitana enfrentan tres limitaciones comunes en su presencia digital. La primera es la **fragmentación de la información**: los pacientes deben llamar por teléfono para conocer disponibilidad de médicos, especialidades, precios o convenios con seguros públicos, lo que genera carga operativa en recepción y una experiencia deficiente. La segunda es la **ausencia de canales digitales** para solicitud de citas: la mayoría de clínicas de este segmento aún opera por teléfono o WhatsApp, sin un formulario web estructurado que recoja los datos esenciales. La tercera es la **falta de centralización** de la información del cuerpo médico, lo que impide al paciente verificar la formación profesional, el número de Colegio Médico del Perú (CMP) o los horarios de cada especialista.

Estas limitaciones se traducen en cuatro problemas concretos para el negocio: (a) saturación del teléfono central durante las horas pico, (b) pérdida de pacientes que abandonan la búsqueda antes de llamar, (c) errores en la toma de datos por canal telefónico, y (d) imposibilidad de aprovechar canales digitales para difundir convenios con EsSalud y SIS.

El proyecto ClinicaWeb aborda estas limitaciones mediante un sitio web institucional con siete vistas funcionales: una página principal, un directorio médico con tabla filtrable, un catálogo de servicios con calculadora de costos, un formulario de solicitud de cita con cálculo de tarifa en vivo, un formulario de registro de paciente, una página de contacto y un panel de indicadores. La solución es estática en cuanto a infraestructura — para reducir costos y facilitar el despliegue — pero **dinámica en el navegador** gracias a JavaScript, que aporta validación, cálculo, filtrado y manipulación del contenido en tiempo real sin necesidad de un servidor de aplicaciones.

3. Objetivos

3.1 Objetivo general

Desarrollar un sitio web responsivo e interactivo para una clínica de servicios médicos, denominado ClinicaWeb, aplicando los principios modernos de diseño web, maquetación con Flexbox y CSS Grid, animaciones y transiciones CSS, formularios con validación, y una capa de comportamiento en JavaScript (manipulación del DOM, eventos, funciones, arrays y procesamiento de formularios), utilizando exclusivamente HTML5, CSS3 y JavaScript como tecnologías de implementación.

3.2 Objetivos específicos

1. Diseñar una arquitectura de carpetas semánticas que organice el sitio en siete páginas HTML agrupadas por dominio funcional, con un archivo `index.html` raíz como punto de entrada.
2. Implementar un sistema de diseño centralizado en `css/tokens.css` mediante variables CSS, definiendo paleta de colores, tipografía, espaciado, radios, sombras y *breakpoints*.
3. Construir un *layout* responsivo basado en Flexbox y CSS Grid, con tres puntos de quiebre (760 px, 1024 px y 1100 px).
4. Incorporar animaciones (`@keyframes`) y transiciones CSS para enriquecer la experiencia: aparición de modales, despliegue del menú y pulsos de estado.
5. Centralizar los datos del dominio (médicos y especialidades) en **arrays paralelos** dentro de `js/data.js` como única fuente de verdad.
6. Desarrollar una capa de comportamiento en JavaScript que incluya: menú responsivo con botón hamburguesa, ventanas flotantes reutilizables, validación y procesamiento de formularios, calculadoras en tiempo real, tablas renderizadas dinámicamente y un panel de indicadores.
7. Construir cinco tablas HTML accesibles (`caption`, `thead`, `tbody`, `scope`) y estilizarlas con CSS.
8. Verificar el comportamiento JavaScript con pruebas automatizadas y desplegar el sitio de forma reproducible.
9. Documentar el desarrollo en un informe técnico siguiendo el formato académico de la UTP.

4. Marco Teórico

El marco teórico se sustenta en seis pilares conceptuales: HTML5 con semántica web, CSS3 con sistemas de *layout* modernos, diseño responsivo, animaciones y transiciones CSS, JavaScript y manipulación del DOM, y el procesamiento de formularios. A continuación se desarrolla cada uno.

4.1 HTML5 y semántica web

HTML5, estandarizado por el World Wide Web Consortium (W3C) en 2014, introduce elementos semánticos que describen el rol estructural de cada bloque de contenido: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` y `<footer>`. A diferencia de los `<div>` genéricos, estos elementos comunican explícitamente la intención del contenido a navegadores, tecnologías asistivas y motores de búsqueda (MDN Web Docs, 2024). En ClinicaWeb cada una de las siete páginas implementa este patrón semántico, reforzado con atributos ARIA (`aria-label`, `aria-current`, `aria-modal`) donde la semántica nativa no alcanza.

4.2 CSS3, Grid y Flexbox

CSS3 representa la modularización del lenguaje de hojas de estilo en cascada. Dos de sus módulos transformaron la maquetación: **Flexbox** (modelo unidimensional, ideal para alinear elementos en una fila o columna) y **CSS Grid** (modelo bidimensional, ideal para rejillas complejas). Ambos resuelven problemas que históricamente requerían trucos con `float` o posicionamiento absoluto (Frain, 2022). El proyecto usa Flexbox en el encabezado, los botones y los grupos de campos, y CSS Grid en las rejillas de tarjetas, el pie de página y los *layouts* de dos columnas.

4.3 Diseño responsivo y unidades relativas

El diseño responsivo fue definido por Ethan Marcotte (2010) como la combinación de rejillas fluidas, imágenes flexibles y *media queries*. ClinicaWeb define tres *breakpoints* (1100 px, 1024 px y 760 px) y usa unidades relativas (`rem`, `%`, `vw`, `ch`) junto con la función `clamp()` para tipografía fluida. Esto es indispensable considerando que más del 70 % del tráfico web en Perú proviene de dispositivos móviles (IAB Perú, 2023).

4.4 Animaciones y transiciones CSS

CSS distingue dos mecanismos de movimiento. Las **transiciones** interpolan un cambio entre dos estados (por ejemplo, el color de un botón al pasar el cursor) y se declaran con `transition`. Las **animaciones** permiten secuencias de varios fotogramas mediante la regla `@keyframes`, ejecutándose en bucle o bajo demanda. ClinicaWeb emplea transiciones en botones, tarjetas y campos de formulario, y animaciones `@keyframes` para la aparición de los modales (desvanecimiento + escala), el despliegue del menú hamburguesa, el pulso de los indicadores de estado y el crecimiento de las barras del *dashboard*. Se respeta la preferencia `prefers-reduced-motion` para usuarios sensibles al movimiento.

4.5 JavaScript y manipulación del DOM

JavaScript es el lenguaje de programación del lado del cliente que dota de comportamiento a las páginas web. El navegador expone el documento como un árbol de objetos — el **DOM** (Document Object Model) — que JavaScript puede consultar y modificar (MDN Web Docs, 2024). Las operaciones centrales utilizadas en el proyecto son: selección de nodos (`querySelector`), creación y modificación de elementos (`createElement`, `textContent`, `classList`), y la suscripción a **eventos** del usuario mediante `addEventListener` (clics, cambios de campo, envíos de formulario). ClinicaWeb aplica estas operaciones para renderizar tablas, abrir ventanas flotantes, validar formularios y actualizar indicadores en tiempo real.

4.6 Arrays paralelos y funciones de arrays

Un **array paralelo** es una técnica de organización de datos en la que varios arreglos independientes comparten el mismo índice: el elemento en la posición i de cada arreglo describe a la misma entidad. Por ejemplo, `nombres[i]`, `especialidad[i]` y `cmp[i]` describen al mismo médico. Sobre estos datos se aplican **funciones de orden superior** propias de JavaScript: `map` (transformar), `filter` (seleccionar), `reduce` (agregar), `sort` (ordenar) y `forEach` (recorrer). Esta combinación es la base del filtrado de tablas, las calculadoras y las métricas del *dashboard*.

4.7 Validación y procesamiento de formularios

HTML5 introdujo atributos de validación nativos (`required`, `pattern`, `minlength`, `maxlength`, `type`, `min`, `max`) que cubren las verificaciones más comunes (Castro & Hyslop, 2021). Sobre esta base, JavaScript permite un **procesamiento** más rico: validar con mensajes personalizados usando la *Constraint Validation API* (`checkValidity()`, objeto `validity`), prevenir el envío real (`preventDefault()`), calcular valores derivados y mostrar una confirmación al usuario. ClinicaWeb combina ambos niveles en sus tres formularios.

5. Implementación de la Solución

5.1 Arquitectura y estructura de carpetas

El proyecto sigue el patrón de sitio multi-página con carpetas semánticas. Cada sección dispone de su propia subcarpeta con un `index.html`, y la lógica JavaScript vive en una carpeta `js/` compartida. La estructura es la siguiente:

Ruta	Descripción
<code>/index.html</code>	Página de inicio (raíz del proyecto)
<code>/medicos/index.html</code>	Directorio médico con tabla filtrable
<code>/servicios/index.html</code>	Catálogo de especialidades y cotizador
<code>/agenda/nueva-cita.html</code>	Formulario de cita con calculadora de costo
<code>/pacientes/registro.html</code>	Registro de paciente con calculadoras
<code>/contacto/index.html</code>	Contacto y formulario general
<code>/dashboard/index.html</code>	Panel de indicadores en tiempo real
<code>/css/tokens.css</code>	Sistema de diseño (única hoja de estilos)
<code>/js/</code>	Capa de comportamiento (7 archivos JavaScript)
<code>/assets/img/</code>	Recursos gráficos
<code>/.github/workflows/</code>	Workflow de despliegue automático a Pages

5.2 Stack y herramientas

Capa	Tecnología / Justificación
Marcado	HTML5 con elementos semánticos.
Estilos	CSS3: Grid, Flexbox, variables, <i>media queries</i> , transiciones y <code>@keyframes</code> .
Comportamiento	JavaScript <i>vanilla</i> (ES6+): DOM, eventos, arrays, módulos por archivo.
Validación	HTML5 nativa + <i>Constraint Validation API</i> de JavaScript.
Versionado	Git con repositorio remoto en GitHub.
Despliegue	GitHub Pages vía GitHub Actions (<code>static.yml</code>); además, imagen Docker + Nginx para entornos propios.
Pruebas	jsdom (Node.js) para pruebas automatizadas del DOM.
Editor	Visual Studio Code.

5.3 Sistema de diseño (tokens.css)

El archivo `css/tokens.css` es la única hoja de estilos del proyecto, importada por las siete páginas. Concentra todos los *tokens* de diseño en el selector `:root` como variables CSS, lo que permite cambiar la apariencia global modificando una sola declaración. Los *tokens* abarcan colores (en el espacio `oklch`), tipografía, espaciado, radios y sombras. La paleta usa verde clínico como acento, azul como color informativo y una escala neutra de grises; el contraste cumple el ratio AA del estándar WCAG 2.1.

5.4 Inventario de páginas

Página	Componentes principales
Inicio	Hero, 6 tarjetas de especialidades, indicadores, banner, CTA.
Médicos	Tabla de directorio (renderizada y filtrable con JS), tabla de disponibilidad semanal, modal de detalle.
Servicios	Tarjetas destacadas, catálogo (renderizado, filtrable y ordenable con JS), cotizador con <code>reduce</code> .
Agenda	Formulario de cita con validación JS, <i>selects</i> dependientes y calculadora de costo; panel lateral.
Pacientes	Formulario de registro con validación JS, calculadora de edad e IMC.
Contacto	Tarjetas de contacto, tabla de líneas por área, formulario con contador de caracteres.
Panel	<i>Dashboard</i> con KPIs, gráfico de barras, <i>donut</i> y rankings calculados en vivo.

5.5 Capa de datos: arrays paralelos (data.js)

El archivo `js/data.js` centraliza los datos del dominio como arrays paralelos y los expone en `window.CLINICA`, evitando duplicar información a lo largo del sitio. Por ejemplo, los médicos se modelan así:

```
const medNombres = ["Dr. Carlos Mendoza", "Dra. María Torres", /* ... */];
const medEspNombre = ["Cardiología", "Pediatría", /* ... */];
const medCmp = ["45231", "38742", /* ... */];
const medEstado = ["disponible", "disponible", /* ... */];
// medNombres[i], medEspNombre[i], medCmp[i] y medEstado[i]
// describen al MISMO médico i (arrays paralelos).
```

5.6 Comportamiento compartido: menú hamburguesa y modales (ui.js)

El archivo `js/ui.js` se carga en las siete páginas y aporta dos piezas reutilizables. La primera es el **menú hamburguesa**: el botón se inyecta por DOM y, en pantallas menores a 760 px, despliega la navegación como panel animado; se aplica mejora progresiva marcando `html.js` para que, sin JavaScript, el menú siga siendo usable. La segunda es un **sistema de ventanas flotantes (modal)** reutilizable, con funciones `openModal()` y `closeModal()`, cierre con tecla Escape o clic en el fondo, y animación de entrada. Estas piezas cubren el requisito de “menú responsivo y ventanas flotantes”.

5.7 Tablas dinámicas y filtros

En las páginas de Médicos y Servicios, las tablas no son estáticas: se renderizan desde los arrays paralelos con `createElement` y se vuelven a dibujar al aplicar filtros. En Médicos se filtra por especialidad y estado; en Servicios se filtra por convenio y se ordena por precio. Esto evidencia manipulación del DOM y uso de `filter`, `sort` y `forEach`.

5.8 Formularios: validación, calculadoras y selects dependientes

Los tres formularios combinan validación HTML5 con procesamiento JavaScript. Al enviar, se previene el comportamiento por defecto, se valida cada campo con mensajes personalizados y, si todo es correcto, se muestra una ventana flotante de confirmación. Además:

- **Cita médica:** al elegir especialidad se filtran los médicos disponibles (*selects* dependientes) y se calcula el costo estimado en vivo, con recargo del 50 % para consultas urgentes.
- **Registro de paciente:** calcula automáticamente la edad a partir de la fecha de nacimiento e incorpora una calculadora de IMC.

- **Contacto:** muestra un contador de caracteres en vivo sobre el mensaje.

Un ejemplo del cálculo del cotizador de servicios, que suma los precios seleccionados con `reduce`:

```
const total = seleccionados.reduce((suma, i) => suma + precios[i], 0);
```

5.9 Dashboard de indicadores

La página `/dashboard/` es un panel que calcula y dibuja métricas en tiempo real a partir de los arrays: indicadores clave (número de médicos, disponibles, precio promedio e ingreso potencial), un gráfico de barras de precio por especialidad, un *donut* de distribución del estado de los médicos (construido con `conic-gradient` calculado en JS) y rankings de tarifas. Los controles de convenio y orden recalculan todo al instante. Es la pieza que demuestra de forma más vistosa el uso conjunto de arrays paralelos, funciones de arrays, eventos y manipulación del DOM.

6. Resultados y Pruebas

6.1 Validación de código

Las siete páginas HTML y la hoja de estilos se validaron contra los servicios del W3C. Las únicas advertencias corresponden a propiedades modernas (`backdrop-filter`, `color-mix`, `oklch`) interpretadas correctamente por navegadores actuales.

6.2 Pruebas funcionales de formularios

Se verificaron manualmente los escenarios: campos obligatorios vacíos, correo con formato inválido, DNI no numérico, fecha fuera de rango, mensaje demasiado corto y consentimiento sin marcar. En todos los casos la validación combinada (HTML5 + JS) impide el envío y muestra un mensaje claro; con datos válidos, aparece la ventana flotante de confirmación.

6.3 Pruebas automatizadas con jsdom

Se construyó una prueba de humo automatizada con `jsdom` que carga cada página, ejecuta los scripts y verifica las transformaciones del DOM. La batería comprueba: inyección del menú hamburguesa, render de tablas desde arrays, funcionamiento de los filtros, cálculo del cotizador ($120 + 80 = 200$), recargo de urgencia ($120 \rightarrow 180$), cálculo de IMC (24.2), contador de caracteres, apertura de modales y métricas del *dashboard*. **Resultado: 30 de 30 verificaciones exitosas.**

6.4 Pruebas de adaptación responsiva

Viewport	Dispositivo	Comportamiento
375×667	Móvil	Menú hamburguesa, formularios en una columna, tablas con <i>scroll</i> .
768×1024	Tablet	Grillas a dos columnas, tablas en su ancho natural.
1280×800	Desktop	<i>Layout</i> completo, grillas a 3–4 columnas, panel lateral <i>sticky</i> .

6.5 Pruebas de navegación interna

Se verificó que los enlaces internos (menú, breadcrumbs, footer, CTA) sean coherentes y no produzcan errores 404. Las rutas relativas están correctamente ajustadas según la profundidad de cada archivo.

6.6 Despliegue

El sitio se despliega automáticamente en **GitHub Pages** mediante un workflow de GitHub Actions que se ejecuta en cada *push* a la rama principal. La URL pública es <https://heyarnoldo.github.io/clinicaweb/>. Como el sitio es 100 % estático y usa rutas relativas, no requiere configuración adicional. De forma complementaria, el repositorio conserva un **Dockerfile** con Nginx para despliegues en infraestructura propia.

7. Conclusiones

Primera. La combinación de HTML5, CSS3 y JavaScript es suficiente para construir un sitio web institucional completo, accesible, responsivo e interactivo, sin recurrir a frameworks ni a lenguajes del lado del servidor. Esta elección reduce la curva de aprendizaje y elimina dependencias externas.

Segunda. Centralizar los datos en arrays paralelos (una única fuente de verdad) es la decisión de diseño más importante del proyecto: de ella se alimentan las tablas, los filtros, las calculadoras y el *dashboard*, evitando inconsistencias.

Tercera. La manipulación del DOM y las funciones de arrays (`map`, `filter`, `reduce`, `sort`) permiten transformar datos en interfaz de forma declarativa y mantenible.

Cuarta. La validación de formularios en dos niveles (atributos HTML5 + *Constraint Validation API*) ofrece a la vez robustez y una experiencia de usuario clara, con mensajes personalizados y confirmaciones visuales.

Quinta. Las animaciones y transiciones CSS, aplicadas con mesura y respetando `prefers-reduced-motion`, elevan la percepción de calidad sin perjudicar la accesibilidad.

Sexta. El proyecto cumple con los ocho aspectos del desarrollo exigidos por la rúbrica: contenido coherente, diseño responsivo con Flexbox y Grid, tablas y formularios validados y estilizados, animaciones y transiciones CSS, JavaScript con eventos y procesamiento de formularios, arrays paralelos y funciones de arrays, menú responsivo y ventanas flotantes, y manipulación del DOM.

8. Recomendaciones

- **Versionado con Git desde el día uno**, con commits descriptivos y revisión cruzada mediante *pull requests*.
- **Validación continua con W3C** al cierre de cada hito de desarrollo.
- **Pruebas en múltiples navegadores**, prestando atención a propiedades modernas (`backdrop-filter`, `color-mix`, `oklch`).
- **Ampliar la cobertura de pruebas automatizadas** más allá de la prueba de humo, hacia casos límite de cada formulario.
- **Optimización de imágenes** a formato WebP y declaración de `width/height` para evitar saltos de *layout*.
- **Evolución hacia un back-end ligero** (por ejemplo NestJS con PostgreSQL) que persista las solicitudes y envíe correos de confirmación; la arquitectura de carpetas actual facilita esa transición.
- **Accesibilidad como criterio continuo**, auditando con Lighthouse o axe DevTools.

9. Bibliografía

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.1 ed.). APA.

- Castro, E., & Hyslop, B. (2021). *HTML5 and CSS3: Visual QuickStart Guide* (9.1 ed.). Peachpit Press.
- Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7.1 ed.). O'Reilly Media.
- Frain, B. (2022). *Responsive web design with HTML5 and CSS* (4.1 ed.). Packt Publishing.
- Haverbeke, M. (2018). *Eloquent JavaScript* (3.1 ed.). No Starch Press. <https://eloquentjavascript.net/>
- IAB Perú. (2023). *Estudio de usos y actitudes hacia internet en Perú 2023*. Interactive Advertising Bureau Perú. <https://iabperu.com/estudios>
- Marcotte, E. (2010, 25 de mayo). Responsive web design. *A List Apart*. <https://alistapart.com/article/responsive-web-design/>
- MDN Web Docs. (2024). *JavaScript*. Mozilla Developer Network. <https://developer.mozilla.org/es/docs/Web/JavaScript>
- MDN Web Docs. (2024). *Document Object Model (DOM)*. Mozilla Developer Network. https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model
- Organización Panamericana de la Salud. (2022). *Transformación digital de la salud en las Américas: Informe 2022*. OPS/OMS. <https://www.paho.org/es/transformacion-digital>
- W3C Web Accessibility Initiative. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>

10. Anexos

Anexo A · Repositorio y sitio en vivo

El código fuente completo está disponible en GitHub: <https://github.com/HeyArnoldo/clinicaweb>. El sitio desplegado puede visitarse en <https://heyarnoldo.github.io/clinicaweb/>. El repositorio incluye HTML, CSS, los siete archivos JavaScript, el workflow de despliegue y la configuración Docker/Nginx.

Anexo B · Capturas de pantalla representativas

Se recomienda adjuntar capturas de las siete páginas en los tres *viewports* principales (móvil 375 px, tablet 768 px, desktop 1280 px), incluyendo: la página de inicio con el menú activo, el formulario de cita mostrando la calculadora de costo, el modal de confirmación, el menú hamburguesa abierto en móvil y el *dashboard* con sus gráficos.

Anexo C · Distribución de responsabilidades

Integrante	Responsabilidad principal
Adriano Joao Souza Reyna	Página de Inicio y maquetación del hero.
Kevin Asencios Sebastian	Página de Médicos y tabla filtrable.
Yosselyn Aracelly Nuñez Vasquez	Página de Servicios y cotizador.
Renzo Steven Contreras Quezada	Página de Agenda y calculadora de cita.
Jesus Miguel Canchanya Soto	Página de Pacientes y calculadoras de registro.
Italo Johan Barreto Villanueva	Página de Contacto y validaciones.

El sistema de diseño (`tokens.css`), la capa JavaScript compartida (`data.js`, `ui.js`), el *dashboard*, las pruebas automatizadas, el despliegue y la redacción del informe se realizaron de manera

colaborativa.

Anexo D · Lista de verificación de la rúbrica

Aspecto de la rúbrica	Estado
Contenido coherente y de alta calidad	Cumplido
Diseño responsivo (Flexbox y Grid)	Cumplido
Tablas y formularios validados y estilizados	Cumplido
Animaciones y transiciones CSS	Cumplido
JavaScript: eventos, funciones, procesamiento de formularios	Cumplido
Arrays paralelos y funciones de arrays	Cumplido
Menú responsivo y ventanas flotantes	Cumplido
Manipulación del DOM	Cumplido
Informe técnico en formato académico UTP	Cumplido